

Name: Dwijesh Mistry
Class: TY4 Batch: B
Roll No: 38
PRN: 23UF17728CM097
Sub: DWM Practical Lab

EXPERIMENT NO – 3

Aim: To implement and analyze **classification algorithms** namely **Decision Tree** and **Naïve Bayes** using Python and evaluate their performance on a standard dataset.

Introduction: Classification is a supervised machine learning technique used to assign input data into predefined classes. It plays a crucial role in applications such as spam detection, medical diagnosis, image recognition, and fraud detection.

Two widely used classification algorithms are:

- **Decision Tree Classifier:** A tree-structured model that makes decisions based on feature values.
- **Naïve Bayes Classifier:** A probabilistic classifier based on Bayes' theorem with an assumption of feature independence.

These algorithms are simple, efficient, and effective for many real-world problems.

Procedure:

1. Import the required Python libraries such as scikit-learn.
2. Load a standard dataset (Iris dataset).
3. Split the dataset into training and testing sets.
4. Create a Decision Tree classifier and train it using the training data.
5. Predict the class labels for test data using the trained model.
6. Evaluate the Decision Tree model using accuracy and classification report.
7. Create a Naïve Bayes classifier and train it using the same training data.
8. Predict the test data and evaluate the Naïve Bayes model.
9. Compare the performance of both classifiers.

Program Codes:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report
```

```

# -----
# Load Iris Dataset
# -----
iris = load_iris()
X = iris.data
y = iris.target

# -----
# Split Dataset into Training and Testing Sets
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# =====
# Decision Tree Classifier
# =====
print("==== Decision Tree Classifier ====")

dt = DecisionTreeClassifier(criterion="entropy")
dt.fit(X_train, y_train)

dt_predictions = dt.predict(X_test)

print("Accuracy:", accuracy_score(y_test, dt_predictions))
print("Classification Report:\n", classification_report(y_test, dt_predictions))

# =====
# Naïve Bayes Classifier
# =====
print("\n==== Naïve Bayes Classifier ====")

nb = GaussianNB()
nb.fit(X_train, y_train)

nb_predictions = nb.predict(X_test)

print("Accuracy:", accuracy_score(y_test, nb_predictions))
print("Classification Report:\n", classification_report(y_test, nb_predictions))

```

Implementation/Output snap shot:

```
===== Decision Tree Classifier =====
Accuracy: 1.0
Classification Report:
              precision    recall  f1-score   support

     0         1.00      1.00      1.00        19
     1         1.00      1.00      1.00        13
     2         1.00      1.00      1.00        13

 accuracy          1.00          1.00          1.00          45
 macro avg         1.00          1.00          1.00          45
weighted avg         1.00          1.00          1.00          45


===== Naïve Bayes Classifier =====
Accuracy: 0.9777777777777777
Classification Report:
              precision    recall  f1-score   support

     0         1.00      1.00      1.00        19
     1         1.00      0.92      0.96        13
     2         0.93      1.00      0.96        13

 accuracy          0.98          0.97          0.98          45
 macro avg         0.98          0.97          0.97          45
weighted avg         0.98          0.98          0.98          45
```

Conclusion: In this experiment, Decision Tree and Naïve Bayes classifiers were successfully implemented using Python. Both algorithms produced high classification accuracy on the Iris dataset. The Decision Tree classifier provides better interpretability, while the Naïve Bayes classifier offers faster computation and simplicity. The choice of classifier depends on the dataset characteristics and application requirements.

Review Questions:

1. What is a Decision Tree classifier, and how does it work?

Ans: A Decision Tree classifier is a supervised learning algorithm that classifies data by repeatedly splitting it based on feature values. It uses a tree-like structure where:

- Each internal node represents a feature test.
- Each branch represents an outcome of the test.
- Each leaf node represents a class label.

The algorithm selects features using criteria such as Information Gain or Gini Index, aiming to create the most homogeneous subsets. Classification is done by traversing the tree from root to leaf based on feature values.

2. Explain the Naïve Bayes algorithm and its underlying assumptions.

Ans: The Naïve Bayes algorithm is a probabilistic classifier based on Bayes' theorem:

$$P(C | X) = \frac{P(X | C) \cdot P(C)}{P(X)}$$

It assumes that:

- All features are **conditionally independent** given the class.
- Each feature contributes independently to the probability of a class.

Despite its “naïve” assumption, it performs well for many real-world problems, especially with large datasets and text classification.

3. Compare the working principles of Decision Tree and Naïve Bayes classifiers.

Ans:

Aspect	Decision Tree	Naïve Bayes
Model type	Tree-based	Probabilistic
Working principle	Rule-based splitting	Probability estimation
Feature dependency	Can handle dependent features	Assumes independence
Interpretability	High	Low
Speed	Moderate	Very fast
Overfitting	Possible	Less prone

4. What are the different types of Decision Tree splitting criteria?

Ans: The commonly used **splitting criteria** in Decision Trees are:

1. **Information Gain**

Measures reduction in entropy after a split.

2. **Entropy**

Measures impurity or randomness in the dataset.

3. **Gini Index**

Measures the probability of incorrect classification.

4. **Gain Ratio**

Adjusts Information Gain by penalizing high-branch splits.

5. **Chi-Square**

Measures statistical significance between feature and target.